CacheMigrate: Efficient Cache Migration for Load Balancing in Distributed Key-Value Storage Systems

Xianda Meng*, Ammar Hawbani**, Jiangyuan Chen†, Abdulbary Naji†, Liang Zhao*
* School of Computer Science, Shenyang Aerospace University, ShenYang, China
Email: mengxianda1@stu.sau.edu.cn, anmande@ustc.edu.cn, lzhao@sau.edu.cn
† School of Computer Science and Technology, University of Science and Technology of China, Hefei, China
Email: chenjy@mail.ustc.edu.cn, abdulbary@mail.ustc.edu.cn

***Abstract*—Load imbalance in distributed key-value (KV) storage systems, especially under skewed access patterns across racks or clusters, can significantly degrade performance. We present CacheMigrate, a switch-assisted caching and migration framework that leverages programmable Top-of-Rack (ToR) switches for real-time hotspot detection, in-network caching, and per-key migration state tracking. By decoupling data movement from request handling, CacheMigrate minimizes service disruption and ensures strong consistency during live migration. A distributed migration state management design coordinates ToR and spine switches without centralized bottlenecks. We implement CacheMigrate on a P4-programmable switch and evaluate it using realistic workloads. Results show that CacheMigrate consistently delivers higher throughput and better load balance than existing approaches across diverse workload and system configurations.**

***Index Terms*—Key-value storage, load balancing, distributed systems, live migration, programmable switches**

I. Introduction

Large-scale key-value (KV) storage systems serve as the fundamental infrastructure for modern Internet services, including e-commerce and social networks. These systems are optimized for mission-critical operations that demand low-latency access, such as real-time data analytics, distributed machine learning training, and metadata indexing [1]–[3]. To address exponentially increasing I/O demands, geo-distributed multi-cluster architectures have become prevalent. Such architectures achieve scalability and fault tolerance through persistent data sharding [4]–[6]. However, this design introduces a storage-specific challenge: cross-cluster load skew. Imbalanced access patterns during inter-cluster operations can create local hotspots. These hotspots degrade system throughput and cause tail latency spikes, potentially violating Service Level Objectives (SLOs) [7]–[10].

Caching is a fundamental method for load balancing in distributed systems. Theoretical analysis demonstrates that caching the top $O(n \log n)$ hottest objects is sufficient to balance the load across n storage nodes, with the cache size scaling only with the number of storage nodes, not the total object count [11]. This principle is leveraged by NetCache [12] to achieve intra-cluster load balancing via localized caching. However, when scaling to multiple clusters, caching within a single cluster becomes inadequate for addressing inter-cluster load imbalance. DistCache [13] attempts to overcome this by merging caches from multiple clusters into a global cache. Despite this, its hotspot selection strategy leads to suboptimal cache utilization, as moderately popular objects in one cluster may receive more requests than the hottest objects in another cluster. This introduces consistency problems, as hotspots are replicated across clusters, complicating cache management. Therefore, static cache-based solutions like DistCache are insufficient for multi-cluster systems, where dynamic hotspot migration offers a more effective approach.

Current dynamic migration strategies for KV stores, including sourcebased [14] and destination-based [15] migration, face significant performance challenges. Both aim to minimize downtime and ensure continuous query servicing, but each has limitations: source-based migration requires additional data transfers when requested data has not yet been copied to the destination, causing extra latency [16], [17]; destination-based migration incurs round-trip delays to fetch unmigrated data. Hybrid approaches leverage both source and destination nodes to improve performance, but often add client-side tracking complexity and consistency-maintenance overhead [18]. Achieving efficient migration with minimal disruption remains a critical challenge.

This paper introduces CacheMigrate, a novel solution addressing load-balancing challenges in multi-cluster storage systems. By leveraging programmable switches, CacheMigrate integrates a high-speed in-network caching layer within the data plane. This approach enables several critical functionalities. Primarily, it serves as a real-time hotspot detector during migration initiation, identifying frequently accessed keys directly within the switch pipeline. Offloading this responsibility from the control plane allows CacheMigrate to trigger migrations swiftly and with minimal overhead, without burdening the source nodes.

A key feature of CacheMigrate is the decoupling of data movement from request servicing during migration. The system embeds per-key migration-state awareness directly into the data plane, enabling dynamic adjustments to the behavior of switches based on the progress of migration. For example, switches can reroute incoming requests or buffer updates as needed, ensuring that migration proceeds with minimal disruption to service and maintaining system correctness throughout

*Corresponding author: Ammar Hawbani (e-mail: anmande@ustc.edu.cn)

the process.

To further enhance migration efficiency, CacheMigrate implements a distributed migration state management framework. This architecture assigns specific responsibilities to each component throughout the migration lifecycle. The source rack initiates migration by identifying and marking hot data, while the destination rack manages the finalization of migration. Spine switches play a crucial role by coordinating routing updates and ensuring that requests are consistently redirected. This distributed design offers scalable, fine-grained tracking of migration states on a per-key basis, effectively avoiding the performance bottlenecks commonly associated with centralized coordination.

In summary, we make the following contributions.

- We propose CacheMigrate, a novel solution for dynamic load balancing in multi-cluster storage systems. By leveraging programmable switches, CacheMigrate integrates the data and control planes to enable real-time migration decisions and efficient resource coordination (§III-A).
- We design and implement a system that tightly couples in-network caching with distributed migration state management on emerging programmable switch hardware. This integration minimizes service disruption and ensures data consistency throughout the migration process (§III-E).
- We build a working prototype using Barefoot Tofino switches and commodity servers integrated with TommyDS. Our evaluation shows that CacheMigrate achieves near-linear scalability with the number of racks, while incurring minimal overhead from its cache coherence protocol (§IV).

To comprehensively detail CacheMigrate, we organize the paper as follows: Section II reviews the background and motivation, Section III presents the system design, Section IV reports experimental evaluation, Section V surveys related work, and Section VI concludes with key findings and future directions.

II. Background & Motivation

A. Caching & Migration Strategies

Caching is a fundamental technique for load balancing in distributed storage systems. It improves latency and reduces backend load by storing the most frequently accessed keys. Prior work shows that caching the top $O(n \log n)$ hottest items across n nodes is enough for near-optimal balance, regardless of dataset size [11]. This works because real workloads are highly skewed, with a small fraction of data serving most requests [19]. Systems like NetCache [12] use localized caching and compact metadata to balance intra-cluster loads, scaling with cluster size rather than dataset cardinality. Caching only a few thousand objects can balance hundreds of nodes, even under skewed workloads such as Zipfian-0.9 and Zipfian-0.99.

However, static caching has limitations in multi-cluster systems. Global-local popularity divergence occurs when moderately popular items in busy clusters create more load than globally popular items elsewhere. Cache efficiency also drops when per-cluster caches are merged into a global hierarchy, lowering hit rates by over 25% [20]. Static caching must therefore evolve into dynamic, adaptive mechanisms.

In distributed KV stores, dynamic re-sharding helps balance load, improve locality, and scale as clusters grow. Live migration moves data without interrupting requests, mitigating hot spots [15], [21]. Source-based migration serves requests from the source while incrementally copying data; destination-based migration lets the destination fetch data on demand. Both maintain availability but trade off latency: source-based delays replication, destination-based suffers round-trip fetches. Hybrid strategies [22], [23] combine the two, but require real-time metadata for ownership tracking, adding overhead and complexity. These issues intensify in multi-cluster environments. Workloads can shift across regions, requiring rapid data reallocation [4]. Centralized migration control becomes a bottleneck, slowing adaptation to changing demand.

B. Load Balancing in Multi-cluster Storage

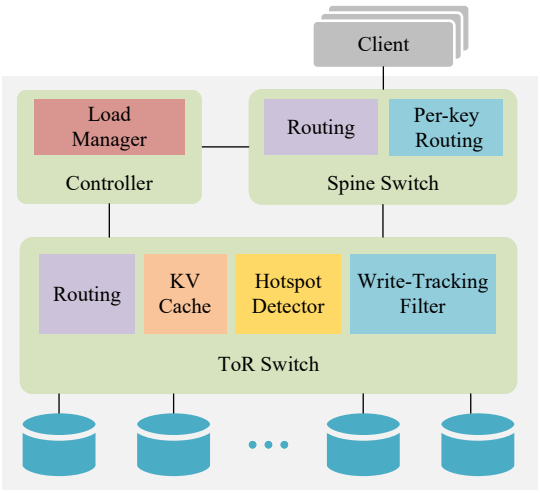
Multi-cluster load balancing is becoming increasingly crucial as modern distributed storage systems extend across geographically dispersed clusters serving diverse user populations [24], [25]. In such environments, request patterns are highly variable and can change unpredictably [26]. A moderately popular key in one cluster may generate more load than the globally hottest key in another, leading to significant cross-cluster imbalances if static policies are maintained. Traditional solutions, like static in-cluster caching and periodic rebalancing, fail to effectively address these dynamics, resulting in degraded performance and increased tail latency.

To overcome these issues, dynamic strategies are needed that can adapt to workload changes in real time without incurring excessive coordination overhead. To this end, we propose distributed migration state management as a key design principle underlying our work. By decentralizing migration control, CacheMigrate allows each cluster to independently monitor progress, trigger state transitions, and coordinate updates via programmable switches in the data plane, thereby achieving faster and more efficient load balancing. This decentralized approach allows the system to dynamically redistribute hot data, reducing reliance on control-plane interventions and significantly improving responsiveness and efficiency for cross-cluster load balancing.

III. CacheMigrate Design

A. System Architecture

Fig 1a shows the per-cluster architecture of CacheMigrate. Each Top-of-Rack (ToR) switch has in-network caching, real-time hotspot detection, and per-key migration state tracking. This design allows dynamic coordination through spine-level routing and control-plane orchestration. CacheMigrate is implemented as an IP-based L3 protocol, and its packet format is illustrated in Fig 1b.



(a) Per Cluster Architecture.

Client Packets:

ETH	IP	OP	SEQ	KEY	VALUE

Migration Data Packets:

ETH	IP	OP	SEQ	KEY	VALUE	MIG ID	SRC ID	DST ID	C IDX

(b) CacheMigrate Network Protocol.

Fig. 1: System Architecture.

a) ToR switch: The ToR switch handles three core functions to support seamless migration: (1) It maintains a KV cache to serve client requests quickly and hide migration delays; (2) It includes a hotspot detector that identifies frequently accessed keys in real time, enabling fast migration when triggered; (3) It implements a Write-Tracking filter to track updates during migration and ensure write consistency. Together, these functions make the ToR switch essential for efficient, transparent migration in CacheMigrate.

b) Spine switch: The spine switch updates routing entries once migration completes, ensuring that client requests are forwarded to the correct destination servers. It maintains global routing consistency across the network and supports load balancing by evenly distributing requests across clusters. By dynamically adapting to migration status, it ensures efficient and reliable request routing.

c) Controller: The controller, or Load Manager, monitors system-wide load metrics from all ToR switches. It detects imbalances and triggers migrations to redistribute load. Based on real-time statistics, it selects source racks and issues migration commands. Its global view and dynamic decision-making allow CacheMigrate to scale effectively under varying workloads.

d) Servers & clients: Storage servers store KV data and transfer selected keys during migration under ToR switch control, without handling hotspot detection or key selection. Destination servers validate data and signal completion. Clients use the CacheMigrate library to send commands and infer migration status by comparing response sources with original destinations, without explicit coordination.

In our design, each system layer is assigned responsibilities based on its capabilities. The control plane makes load balancing decisions and initiates migrations. The source cluster selects migration content and triggers the process, while the destination cluster ensures data consistency and commits state transitions. Spine switches handle dynamic routing updates, ensuring accurate redirection of client requests throughout migration. This division of responsibilities enables efficient, consistent, and low-overhead migration without relying on centralized orchestration for every step.

B. Challenges and Goals

1) Decentralizing migration control: In large-scale systems, centralized tracking of migration progress becomes a bottleneck. The control plane must maintain per-key migration state across thousands of storage nodes, which limits scalability and responsiveness. Moreover, programmable switches have limited on-chip resources, making it impractical to store large volumes of state directly in the data plane.

To address this, CacheMigrate adopts a decentralized approach to migration control. Instead of relying on a central controller, each cluster manages per-key migration state locally. The source and destination coordinate with their ToR switches to track progress and update status. Routing updates are handled by spine switches based on key states, allowing the system to support concurrent migrations without overloading the control plane. This design reduces coordination overhead and enables scalable, fine-grained migration with minimal switch resource usage.

2) Service disruption during migration: A key challenge in live migration is avoiding service disruption when a key is in transit. During this period, client requests may arrive at either the source or destination cluster, risking increased latency, incorrect routing, or request failure. Existing systems often require coordination between endpoints or rely on clients to retry failed requests, resulting in inconsistent performance.

CacheMigrate addresses this challenge by leveraging in-switch caching to isolate client request handling from the underlying migration process. Specifically, the ToR switch continues to serve queries from its cache even as the backend migrates data between clusters. This allows client requests to be processed without delay, regardless of whether the migration has completed. The switch updates its routing only after the destination confirms the migration is complete. As a result, migration can proceed in the background without interrupting service, enabling seamless and transparent request handling throughout the migration lifecycle.

3) Data consistency: Maintaining consistency during migration is challenging when reads and writes continue. Conventional systems risk violations if updates hit both source and destination, or if stale data is served due to delayed synchronization. These risks grow in distributed settings with higher latency and coordination costs.

CacheMigrate adds complexity by serving queries from in-switch caches during migration. While this avoids service disruption, it risks stale reads and conflicting writes if cached data diverges from the backend.

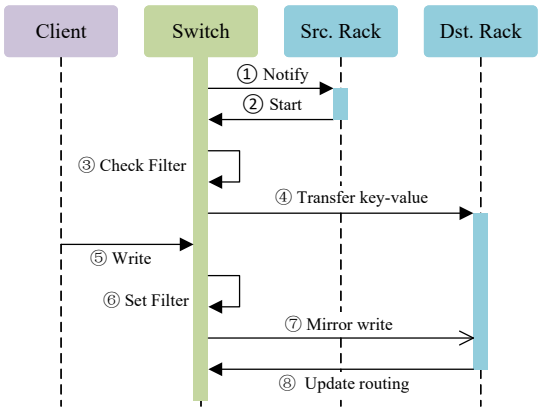


Fig. 2: Migration Flow.

To address this, CacheMigrate tracks migration state per key and enforces strict state transitions. Writes are redirected to the correct backend according to the migration phase. Switches permit cache reads only for consistent keys, and invalidate entries on state changes. This prevents duplicate writes and stale responses, ensuring strong consistency throughout migration.

C. Overview of Migration Workflow

In the previous sections, we have outlined the roles of key components involved in the CacheMigrate system, including the storage servers, clients, and control plane. These components work together to facilitate seamless data migration. To provide a clearer understanding of how these components interact during the migration process, we now present an overview of the migration workflow.

Fig 2 illustrates the basic KV migration process in CacheMigrate. Most migration tasks are handled by the destination rack to minimize load on the source rack. When the control plane detects that a rack is overloaded, it triggers a migration request (Step ①). The source rack then informs its ToR switch to initiate the migration. The source server does not select migration content, further reducing its workload (Step ②).

The ToR switch in the source rack selects the keys to migrate and prepares to send them to the destination rack (Step ④). Before sending, it checks an internal Write-Tracking filter to decide whether a key is eligible for migration (Step ③). Once the destination rack receives the keys, it updates the routing table via the switch to ensure future requests are directed to the new location (Step ⑧).

Importantly, clients are unaware of the migration. During the process, if a write request targets a key being migrated (Step ⑤), the Write-Tracking filter logs it to prevent further migration of that key (Step ⑥). The switch also mirrors the write request to the destination rack (Step ⑦). After receiving the mirrored request, the destination rack completes the migration and takes over ownership of the data (Step ⑧).

D. Migration State Management

1) Load manager: The Load Manager is responsible for continuously monitoring cluster-level load and initiating migration procedures when necessary. Specifically, it periodically collects per-port counters from each ToR switch to estimate metrics such as request rates, cache hit ratios, and heavy-hitter statistics. Based on these aggregated measurements, it computes a load index for each rack and determines whether sustained overload conditions are present.

Global cache hit rate is a key indicator of the system's capacity to absorb request load without resorting to migration. For each time interval t, we define the global cache hit count $H(t)$ as

$$H(t) = \sum_{i=1}^{N} \mathrm{Hits}_i(t). \tag{1}$$

The per-second cache hit rate is given by

$$\mathrm{HitRate}(t) = \frac{H(t)}{\sum_{i=1}^{N} \mathrm{TotalRequests}_i(t)}. \tag{2}$$

When the global hit rate remains high, the majority of requests are satisfied directly by ToR caches, reducing backend load and diminishing the urgency of migration. Conversely, sustained decreases in global hit rate signal that cache resources are insufficient to accommodate shifting access patterns, thereby increasing the risk of server overload. CacheMigrate monitors global hit rate trends over time and dynamically adjusts migration policies accordingly. In particular, lower hit rates trigger more aggressive migration, both in terms of the frequency of migration events and the volume of keys selected for transfer. This adaptive strategy ensures that migration bandwidth is allocated where it can most effectively alleviate load imbalance.

The load index L is defined as

$$L = \alpha \cdot \frac{b(t) + p(t)}{\Delta t} + \beta \cdot \mathrm{AverageTrend}, \tag{3}$$

where α and β are configurable weighting factors (e.g., α = 0.9, β = 0.1), $b(t)$ and $p(t)$ denote bytes and packets observed in the most recent interval, and $\mathrm{AverageTrend}$ measures the mean incremental rate of change across recent intervals. This formulation allows the system to capture both instantaneous overload and sustained trends when making migration decisions.

When the load index exceeds a configurable threshold, the Load Manager notifies the source rack to initiate migration. Importantly, it does not participate in selecting which keys to migrate; this decision is delegated entirely to the source rack, which relies on its local heavy-hitter detection and cache metadata to identify suitable migration candidates. This design decouples global load monitoring from per-rack migration planning, reducing the complexity of the control plane and improving scalability.

2) Write-Tracking filter: we use a probabilistic Bloom filter (BF) on each ToR switch to track keys modified during migration. This compact structure stores millions of entries with only a few hundred kilobytes, avoiding the high memory cost of explicit per-key state. When a write occurs after migration starts, the key is inserted into the filter; during

migration, a match causes the system to skip transferring that key to prevent stale data propagation, while still mirroring and routing writes for correctness (§III-E). Although BF have a false positive probability $p \approx \left(1 - e^{-\frac{kn}{m}}\right)^k$, we treat matches conservatively to ensure correctness. Our P4-based implementation allocates 2^{20} bits and three hash functions, yielding a false positive rate below 1% in typical workloads, with all operations running at line rate without control-plane involvement.

3) Distributed state management: Centralized coordination is often a scalability bottleneck in distributed systems. Maintaining per-key migration progress globally imposes high coordination overhead and increases latency, especially under dynamic or concurrent migrations. To address this, CacheMigrate adopts a distributed approach, assigning migration responsibilities to different components based on their roles and visibility.

The control plane, which has a global view of cluster-wide load conditions, is responsible for initiating migrations (§III-D1). The source cluster triggers the migration process, since it owns the initial state and can identify candidate data. To assist this process, the source ToR switch employs a Write-Tracking filter to monitor write activity and mark hot keys for migration (§III-D2). This ensures that frequently updated data is prioritized and captured accurately. The destination cluster, in turn, completes the migration by validating data consistency and signaling readiness to serve future requests—marking the natural end of the migration.

Finally, routing updates are handled by spine switches. As backbone nodes that interconnect all clusters, spine switches are ideally positioned to perform consistent path redirection across the system. Once a migration completes, spine switches update their forwarding logic to route subsequent client requests to the new data location. This division of responsibilities enables scalable, fine-grained coordination without relying on global synchronization.

E. Query Handling During Migration

When a key is selected for migration, there is a transition period from the moment the source rack initiates migration until the spine layer updates its routing table to forward future requests to the new server. During this period, CacheMigrate uses the cache layer on the ToR switch to serve all related queries. This ensures that clients experience no visible disruption or downtime. Algorithm 1 shows the pseudocode for query handling during migration.

a) Read requests: When a client sends a read request for a migrating key, the system first checks the ToR cache. If the key is found in the cache, the switch replies directly. This avoids backend involvement and reduces latency. If the key is not in the cache, the request is sent to the source server. The source server remains responsible for the key until migration finishes. After migration completes and the routing is updated, new read requests go to the destination server (Algorithm 1, lines 2-4). This mechanism ensures uninterrupted read service and isolates the client from migration details.

Algorithm 1 Query Handling During Migration

Input: Request `req`, Key-Value cache `cache`, Write-Tracking filter `filter`
Output: Correct routing and consistency during migration

```
1:  if req.key exists in cache and is valid then
2:    if req.op == GET then
3:      req.value ← cache[req.key]
4:      return req
5:    else if req.op == SET then
6:      cache[req.key].valid ← false
7:      routing(req, source)
8:      mirror(req, destination)
9:      filter.set(req.key)
10:   end if
11: end if
```

b) Write requests: Write operations on keys under migration require consistency protection. Before the routing is updated at the spine switches, the system always uses the source rack as the authoritative replica. When a write request arrives, the data plane performs two actions. It forwards the request to the source server, which processes and acknowledges the write. Simultaneously, it mirrors the request to the destination server to maintain data consistency. The key is also added to the Write-Tracking Filter. This filter tracks keys that are modified during migration. During the transfer phase, the migration engine checks the filter. If the key is marked, the system skips migrating it to avoid overwriting newer data (Algorithm 1, line 5-9). This conservative approach ensures that no stale data reaches the destination, even when writes occur during migration.

F. Cache Consistency During Migration

During key migration, ensuring cache consistency is critical to maintaining correct system behavior. Clients may issue read or write requests to a key while it is being moved between racks. If these operations are not properly coordinated, they can lead to consistency violations. In particular, we focus on two common scenarios: Read-After-Write (RAW) and Write-After-Write (WAW) conflicts. These conflicts must be carefully handled to prevent stale reads or inconsistent writes.

a) RAW: A client may issue a write followed immediately by a read to the same key. Since all writes are forwarded to the source server and mirrored to the destination, the source remains the authoritative node until the migration completes. To ensure the latest value is returned, all reads during migration are routed to the source server, even if the destination has already received mirrored updates.

b) WAW: For this scenario, a client performs multiple writes to the same key during an ongoing migration. CacheMigrate preserves write ordering at the source, which serializes and acknowledges all writes as the primary replica. Mirrored

writes are sent to the destination in the same order. Additionally, the Write-Tracking filter ensures that no stale state from the source is copied over after these writes, preserving the correct final state at the destination.

IV. Experiments & Evaluation

A. Experimental Setup

We have implemented a full-stack prototype of CacheMigrate to evaluate its feasibility and performance. The system consists of cache-enabled programmable switches, a centralized controller, and a set of storage servers and clients.

Our experiments are conducted on a dedicated cluster of commodity servers connected via a high-speed data center network. Each server is equipped with an Intel Xeon Silver 4310 CPU (12 cores @ 2.10 GHz), 128 GB DDR4 memory, and a Mellanox ConnectX-5 NIC (100 Gbps) with DPDK [27] enabled for kernel-bypass packet processing. The network fabric includes Mellanox SN2700 (100 GbE) switches and one Barefoot Tofino [28] programmable switch with a switching capacity of 6.5 Tbps. All servers run `Ubuntu 20.04` with `Linux kernel 5.15`.

To maximize testbed utilization, we partition both the programmable switch and servers into multiple logical components, running real switch data-plane programs and server software simultaneously. Specifically, the Barefoot Tofino switch is configured to emulate 32 spine switches, 32 leaf switches, and 32 client-access switches, while the servers act as storage nodes, controller instances, and clients in parallel. Each switch cache stores 128 KV pairs by default, with both keys and values being 128-bit in size.

We deploy `TommyDS` [29] as the backend KV store, configuring each physical server to emulate 32 racks, with each rack hosting 32 server instances. This setup allows the testbed to scale to a large number of logical servers while using only a limited number of physical machines, ensuring realistic request handling and migration behavior under high concurrency. We also rate-limit `TommyDS` to prevent individual servers from becoming bottlenecks and to emulate realistic aggregate throughput.

Workloads are generated using the Yahoo! Cloud Serving Benchmark (YCSB) [30] with different hotspot intensities, characterized by Zipfian distribution parameters ($\theta = 0.9$, 0.95, and 0.99). These workloads allow us to evaluate system performance under varying degrees of load skew.

B. Baselines

We compare CacheMigrate against three representative approaches: **DistCache** [13], a hierarchical caching framework that partitions cache space across multiple layers to balance load and reduce backend traffic. **NetCache** [12], a switch-based in-network caching system that uses programmable switches to store hot objects and directly serve client requests, thereby reducing load on backend servers. **NoCache**, a baseline without any caching, where all requests are sent directly to backend servers, serving as the lower bound for performance.

C. Results & Analysis

1) Throughput with different key access distributions: Fig 3a compares the throughput of the four approaches under varying load skewness. CacheMigrate maintains higher throughput by dynamically redistributing key placements to adapt to different skew levels. As inter-rack load skew increases, NetCache suffers from rack-level imbalance, leading to a slight throughput drop. DistCache achieves higher throughput by caching a larger set of objects, partially mitigating the impact of skew.

2) Latency: As shown in Fig. 3b, across the throughput range of 1.5 to 11 MQPS, CacheMigrate maintains a low P99 latency, comparable to NetCache and DistCache. In contrast, NoCache exhibits significantly higher latency and reaches saturation at 2 MQPS. This indicates that CacheMigrate effectively mitigates latency spikes under increasing load by leveraging cache migration, whereas the absence of caching leads to early saturation and degraded tail performance.

3) Imbalance: Using Equation (3), we compute each rack's load index and define the Imbalance Ratio as the maximum load divided by the average. As shown in Fig. 3c, CacheMigrate consistently achieves the lowest imbalance: 2.13 under uniform traffic and 2.31 under highly skewed Zipf-0.99 workloads, whereas NetCache and NoCache reach 3.9. This demonstrates that CacheMigrate effectively balances load across racks, even under highly skewed access patterns.

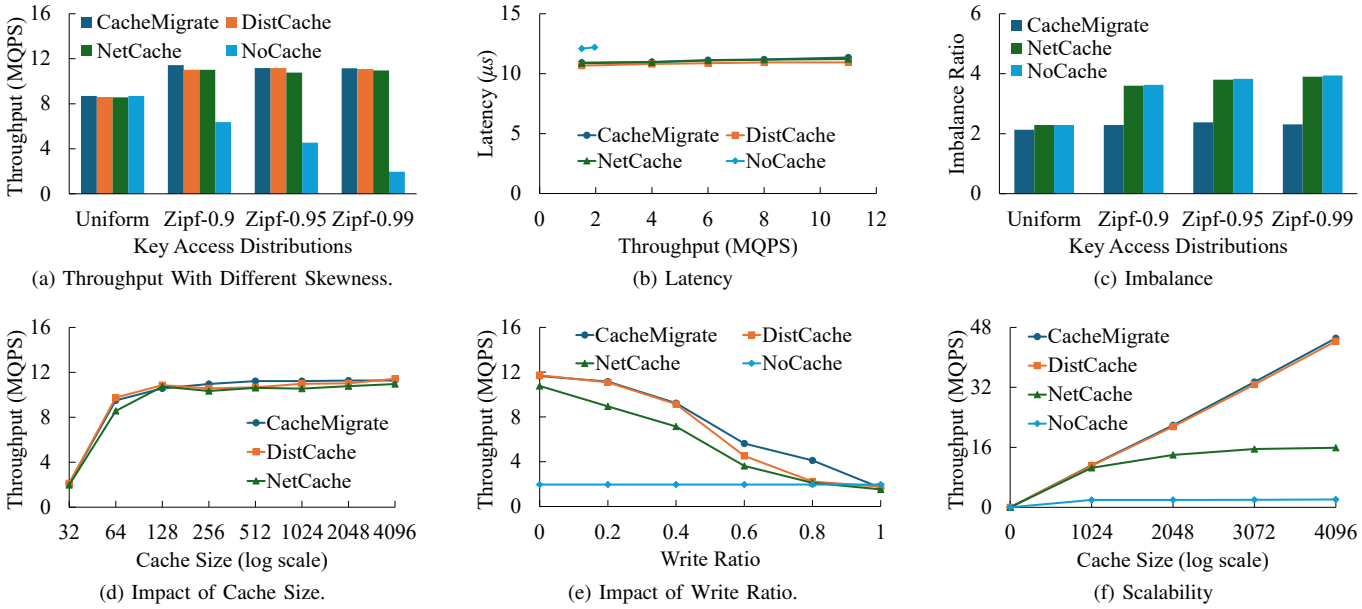
4) Impact of cache size: Fig. 3d shows the throughput of the three mechanisms under different switch cache sizes. Larger caches absorb more hotspot requests, reducing backend load. CacheMigrate further enhances cache utilization by dynamically reallocating keys during migration, making full use of additional capacity to balance load across racks.

5) Impact of writ ratio: As shown in Fig. 3e, the system throughput decreases gradually as the write ratio increases. This trend is consistent across all compared mechanisms, with CacheMigrate maintaining a performance advantage due to its ability to effectively handle write-induced cache invalidations.

6) Scalability: Fig. 3f shows the scalability of the different caching schemes as the number of cached hot objects increases. CacheMigrate and DistCache scale nearly linearly, demonstrating effective utilization of additional cache capacity. In contrast, NetCache grows sublinearly and NoCache shows minimal improvement. These results highlight that CacheMigrate can efficiently leverage larger cache sizes to sustain high throughput under increasing workloads.

7) Impact of migrateion size: We evaluate the impact of migration size on system throughput by varying the ratio of migrated data to the total switch cache capacity from 0 to 1. As shown in Fig. 4, throughput increases with moderate migration (up to about 0.6 ratio) as load becomes more balanced, but declines when the ratio exceeds 0.6 due to higher migration overhead. This result indicates an optimal migration size that balances load redistribution and migration cost.

8) Hardware resources: As shown in Table I, the hardware resources encompass memory blocks, lookup tables, and logic units required to implement the data plane pipeline. These



(a) Throughput With Different Skewness. (b) Latency (c) Imbalance

(d) Impact of Cache Size. (e) Impact of Write Ratio. (f) Scalability

Fig. 3: System Performance.

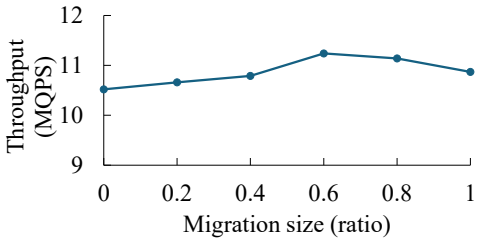


Fig. 4: Impact of Migration Size

TABLE I: Hardware Resource Usage Summary

SRAMs	TCAMs	Map RAMs	Logical Tables
124	13	50	65

resources determine the design's capacity, flexibility, and performance characteristics across different processing stages.

9) Protocol efficiency: To evaluate CacheMigrate, we first consider the Ethernet/IPv4 overhead, which consists of 14 bytes for the Ethernet header and 20 bytes for the IPv4 header. The CacheMigrate-specific headers are 42 bytes for `Client Packet` and 52 bytes for `Migration Data Packet`(Fig 1b).

For a `Client Packet` carrying 42 bytes of payload, the total packet length is 76 bytes, yielding a protocol efficiency of $\eta_{KV} \approx 55.3\%$. For a `Migration Data Packet` carrying 52 bytes of payload, the total packet length is 86 bytes, giving an efficiency of $\eta_{Migrate} \approx 60.5\%$.

By packing multiple control fields into a compact header, CacheMigrate reduces protocol overhead and achieves high throughput for rack-level key migration.

V. Related Work

Cache systems. Memcached [31] and Redis [32] Cluster improve performance for read-heavy workloads but face challenges in write-heavy scenarios and in maintaining consistency across distributed deployments [33], [34]. These limitations motivate more advanced cache management and migration strategies to balance load and optimize data locality.

Cache migration mechanisms. Cache migration moves data between nodes to improve hit rates and balance load, but often incurs overhead that can reduce throughput. Prior work focuses on hot-object migration or consistency maintenance. CacheMigrate builds on these ideas with more efficient, low-overhead migration.

Programmable data plane. Programmable data planes such as P4 [35] and Barefoot Tofino [28] enable custom, fine-grained load balancing directly in the network. This approach optimizes traffic distribution and reduces congestion. CacheMigrate leverages these capabilities to enhance cache migration and request forwarding efficiency.

VI. Conclusion

We present CacheMigrate, a switch-assisted caching and migration framework that dynamically adapts to workload skewness in distributed KV stores. Leveraging programmable switches for hot-object caching and fine-grained migration control, CacheMigrate improves throughput and load balance across diverse workloads. Our evaluation shows that in-network coordination effectively mitigates rack-level imbalances while maintaining strong consistency. Looking ahead, we plan to offload more coordination and state management into the switch data plane to reduce control-plane overhead and enable faster, fully in-network adaptation to workload dynamics. In addition, we will investigate the integration of

security mechanisms and enhanced fault-tolerance strategies to ensure reliable operation under node or switch failures and maintain data integrity during migration. Furthermore, we plan to evaluate the energy consumption and power overhead introduced by migration operations, additional packet processing, and Bloom Filter queries to provide a more comprehensive assessment of system efficiency.

ACKNOWLEDGMENT

This paper was supported by Shenyang Aerospace University Talent Research Start-up Fund under Grant 502/120423005. LLM was used for proofreading.

REFERENCES

[1] A. Anwar, Y. Cheng, H. Huang, J. Han, H. Sim, D. Lee, F. Douglis, and A. R. Butt, "Customizable scale-out key-value stores," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 9, pp. 2081–2096, 2020.

[2] T. Wang, A. Moody, Y. Zhu, K. Mohror, K. Sato, T. Islam, and W. Yu, "Metakv: A key-value store for metadata management of distributed burst buffers," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2017, pp. 1174–1183.

[3] J. Zhao, J. Liu, H. Wang, C. Xu, W. Gong, and C. Xu, "Measurement, analysis, and enhancement of multipath tcp energy efficiency for datacenters," *IEEE/ACM Transactions on Networking*, vol. 28, no. 1, pp. 57–70, 2020.

[4] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," *ACM SIGOPS operating systems review*, vol. 41, no. 6, pp. 205–220, 2007.

[5] R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H. C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab *et al.*, "Scaling memcache at facebook," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013, pp. 385–398.

[6] Z. Zhou, L. Li, P. Zhao, P.-A. Heng, and W. Gong, "Class-conditional sharpness-aware minimization for deep long-tailed recognition," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 3499–3509.

[7] Y.-J. Hong and M. Thottethodi, "Understanding and mitigating the impact of load imbalance in the memory caching tier," in *Proceedings of the 4th annual Symposium on Cloud Computing*, 2013, pp. 1–17.

[8] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[9] A. Naji, X. Wang, A. Hawbani, A. Ghannami, L. Zhao, X. Xu, and W. Zhao, "Netmod: Toward accelerating cloud ran distributed unit modulation within programmable switches," *IEEE Transactions on Computers*, vol. 74, no. 2, pp. 665–677, 2025.

[10] A. Naji, X. Wang, P. Liu, A. Hawbani, L. Zhao, X. Xu, and F. Miao, "Netcrc-nr: In-network 5g nr crc accelerator," *IEEE Transactions on Computers*, vol. 74, no. 4, pp. 1418–1430, 2025.

[11] B. Fan, H. Lim, D. G. Andersen, and M. Kaminsky, "Small cache, big effect: Provable load balancing for randomly partitioned cluster services," in *Proceedings of the 2nd ACM Symposium on Cloud Computing*, 2011, pp. 1–12.

[12] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, and I. Stoica, "Netcache: Balancing key-value stores with fast in-network caching," in *Proceedings of the 26th symposium on operating systems principles*, 2017, pp. 121–136.

[13] Z. Liu, Z. Bai, Z. Liu, X. Li, C. Kim, V. Braverman, X. Jin, and I. Stoica, "{DistCache}: Provable load balancing for {Large-Scale} storage systems with distributed caching," in *17th USENIX Conference on File and Storage Technologies (FAST 19)*, 2019, pp. 143–157.

[14] J. Kang, L. Cai, F. Li, X. Zhou, W. Cao, S. Cai, and D. Shao, "Remus: Efficient live migration for distributed databases with snapshot isolation," in *Proceedings of the 2022 international conference on management of data*, 2022, pp. 2232–2245.

[15] C. Kulkarni, A. Kesavan, T. Zhang, R. Ricci, and R. Stutsman, "Rocksteady: Fast migration for low-latency in-memory storage," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 390–405.

[16] P. Kokkinos, D. Kalogeras, A. Levin, and E. Varvarigos, "Survey: Live migration and disaster recovery over long-distance networks," *ACM Computing Surveys (CSUR)*, vol. 49, no. 2, pp. 1–36, 2016.

[17] Y.-S. Lin, S.-K. Pi, M.-K. Liao, C. Tsai, A. Elmore, and S.-H. Wu, "Mgcrab: transaction crabbing for live migration in deterministic database systems," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 597–610, 2019.

[18] N. Afonso, M. Bravo, and L. Rodrigues, "Combining high throughput and low migration latency for consistent data storage on the edge," in *2020 29th International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2020, pp. 1–11.

[19] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*, 2012, pp. 53–64.

[20] D. S. Medeiros, H. N. Cunha Neto, M. A. Lopez, L. C. S. Magalhães, N. C. Fernandes, A. B. Vieira, E. F. Silva, and D. M. F. Mattos, "A survey on data analysis on large-scale wireless networks: online stream processing, trends, and challenges," *Journal of Internet Services and Applications*, vol. 11, pp. 1–48, 2020.

[21] A. Katsarakis, Y. Ma, Z. Tan, A. Bainbridge, M. Balkwill, A. Dragojevic, B. Grot, B. Radunovic, and Y. Zhang, "Zeus: locality-aware distributed transactions," in *Proceedings of the Sixteenth European Conference on Computer Systems*, 2021, pp. 145–161.

[22] E. Vasilakis, V. Papaefstathiou, P. Trancoso, and I. Sourdis, "Hybrid2: Combining caching and migration in hybrid memory systems," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 649–662.

[23] V. Young, Z. A. Chishti, and M. K. Qureshi, "Tictoc: Enabling bandwidth-efficient dram caching for both hits and misses in hybrid memory systems," in *2019 IEEE 37th International Conference on Computer Design (ICCD)*. IEEE, 2019, pp. 341–349.

[24] D. Bachar, "Optimizing service selection and load balancing in multi-cluster microservice systems," Master's thesis, Reichman University (Israel), 2023.

[25] P. Duraisamy, W. Xu, S. Hare, R. Rajwar, D. Culler, Z. Xu, J. Fan, C. Kennelly, B. McCloskey, D. Mijailovic *et al.*, "Towards an adaptable systems architecture for memory tiering at warehouse-scale," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 727–741.

[26] K. Shen, "Request behavior variations," *ACM SIGARCH Computer Architecture News*, vol. 38, no. 1, pp. 103–116, 2010.

[27] DPDK Project, "DPDK — the open source data plane development kit accelerating network performance," https://www.dpdk.org/, 2025, accessed: 2025-07-01.

[28] Intel Corporation, "Explore the Power of Intel® Intelligent Fabric Processors — Tofino," https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch.html#tofino, 2025, accessed: 2025-07-01.

[29] "GitHub - amadvance/tommyds: A C library of hashtables and tries designed to store objects with high performance — github.com," https://github.com/amadvance/tommyds, [Accessed 12-08-2025].

[30] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.

[31] Dormando, "memcached - a distributed memory object caching system — memcached.org," https://www.memcached.org/, [Accessed 01-07-2025].

[32] Redis, "Redis - The Real-time Data Platform — redis.io," https://redis.io/, [Accessed 01-07-2025].

[33] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The google file system," in *Proceedings of the nineteenth ACM symposium on Operating systems principles*, 2003, pp. 29–43.

[34] D. Beaver, S. Kumar, H. C. Li, J. Sobel, and P. Vajgel, "Finding a needle in haystack: Facebook's photo storage," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*, 2010.

[35] O. N. Foundation, "P4 - Open Networking Foundation — opennetworking.org," https://opennetworking.org/p4/, [Accessed 01-07-2025].